Course Title: Cryptography and Network Security Lab	**Course Code**: BCSE309P
Faculty: Prof. Karthika V	**Slot:** L43 + L44
Regno: 23BCE1145	**Name:** S Vishwajith

EX7 – MD5

Code:

```
global A, B, C, D, M_index, shift, K


def convert_string_to_ASCII_hexa(s):
    sa = ""
    for i in s:
        sa += hex(ord(i))[2:].rjust(2, '0')
    return sa


def add_in_hexa(a, b):
    a1 = int(a, 16)
    b1 = int(b, 16)
    res = (a1 + b1) % pow(2, 32)
    return hex(res)[2:].rjust(8, '0')


def circular_left_shift(s, n):
    s1 = bin(int(s, 16))[2:].rjust(32, '0')
    res = s1[n:]+s1[:n]
    return hex(int(res, 2))[2:].rjust(8, '0')



def function_F(b, c, d):
    b1, c1, d1, b1not = int(b, 16), int(c, 16), int(d, 16), ~int(b, 16) & 0xFFFFFFFF
    res = (b1 & c1) | (b1not & d1)
```

```
    return hex(res)[2:].rjust(8, '0')

def function_G(b, c, d):
    b1, c1, d1, d1not = int(b, 16), int(c, 16), int(d, 16), ~int(d, 16) & 0xFFFFFFFF
    res = (b1 & d1) | (c1 & d1not)
    return hex(res)[2:].rjust(8, '0')

def function_H(b, c, d):
    b1, c1, d1 = int(b, 16), int(c, 16), int(d, 16)
    res = b1 ^ c1 ^ d1
    return hex(res)[2:].rjust(8, '0')

def function_I(b, c, d):
    b1, c1, d1not = int(b, 16), int(c, 16), ~int(d, 16) & 0xFFFFFFFF
    res = c1 ^ (b1 | d1not)
    return hex(res)[2:].rjust(8, '0')

def work(round, M):
    global A, B, C, D, M_index, shift, K
    A1 = ""
    for i in range(0, 16):
        if(round == 0):
            A1 = function_F(B, C, D)
        if(round == 1):
            A1 = function_G(B, C, D)
        if(round == 2):
            A1 = function_H(B, C, D)
        if(round == 3):
            A1 = function_I(B, C, D)

        A1 = add_in_hexa(A, A1)
```

```
        A1 = add_in_hexa(M[8*M_index[round][i] : 8*M_index[round][i] + 8], A1)
        A1 = add_in_hexa(K[round][i], A1)
        A1 = circular_left_shift(A1, shift[round][i])
        A1 = add_in_hexa(A1, B)
        A, B, C, D = D, A1, B, C

M_index = [
    [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15],
    [1, 6, 11, 0, 5, 10, 15, 4, 9, 14, 3, 8, 13, 2, 7, 12],
    [5, 8, 11, 14, 1, 4, 7, 10, 13, 0, 3, 6, 9, 12, 15, 2],
    [0, 7, 14, 5, 12, 3, 10, 1, 8, 15, 6, 13, 4, 11, 2, 9]
]

K = [
    [
        "D76AA478", "E8C7B756", "242070DB", "C1BDCEEE",
        "F57C0FAF", "4787C62A", "A8304613", "FD469501",
        "698098D8", "8B44F7AF", "FFFF5BB1", "895CD7BE",
        "6B901122", "FD987193", "A679438E", "49B40821"
    ],
    [
        "F61E2562", "C040B340", "265E5A51", "E9B6C7AA",
        "D62F105D", "02441453", "D8A1E681", "E7D3FBC8",
        "21E1CDE6", "C33707D6", "F4D50D87", "455A14ED",
        "A9E3E905", "FCEFA3F8", "676F02D9", "8D2A4C8A"
    ],
    [
        "FFFA3942", "8771F681", "6D9D6122", "FDE5380C",
        "A4BEEA44", "4BDECFA9", "F6BB4B60", "BEBFBC70",
        "289B7EC6", "EAA127FA", "D4EF3085", "04881D05",
        "D9D4D039", "E6DB99E5", "1FA27CF8", "C4AC5665"
```

```
    ],
    [
        "F4292244", "432AFF97", "AB9423A7", "FC93A039",
        "655B59C3", "8F0CCC92", "FFEFF47D", "85845DD1",
        "6FA87E4F", "FE2CE6E0", "A3014314", "4E0811A1",
        "F7537E82", "BD3AF235", "2AD7D2BB", "EB86D391"
    ]
]
s = input("Enter an input string: ")
sa = convert_string_to_ASCII_hexa(s)

shift = [[7, 12, 17, 22] * 4, [5, 9, 14, 20] * 4, [4, 11, 16, 13] * 4, [6, 10, 15, 21] * 4]

nsa = len(sa) * 4
sa += '8'
n = len(sa) + 16
if(n % 128 == 0):
    pass
else:
    if(n < 128):
        sa += '0'*(128-n)
    else:
        sa += '0'*(128-(n%128))

sa += hex(nsa)[2:].rjust(16, '0')

A = "01234567"
B = "89ABCDEF"
C = "FEDCBA98"
D = "76543210"
```

```
for i in range(0, len(sa), 128):
    print(f"\nBlock {i//128+1}: \"{sa[i:i+128]}\".")
    for j in range(0, 4):
        print(f"Round {j+1}:")
        work(j, sa)
        print(f"A: \"{A}\"," , end=" ")
        print(f"B: \"{B}\",", end=" ")
        print(f"C: \"{C}\",", end=" ")
        print(f"D: \"{D}\".")


print(f"\nFinal MD5 Hash is: \"{A+B+C+D}\".")
```

Output:

Test Case 1:

```
PS C:\Users\vishw\Coding\Cryptography-Sem-6\Week-9> python .\MessageDigest5.py
Enter an input string: They are deterministic

Block 1: "546865792061726520646574657465726d696e69737469638000000000
```

Test Case 3:

```
PS C:\Users\vishw\Coding\Cryptography-Sem-6\Week-9> python .\MessageDigest5.py
Enter an input string: he he he
```